Bank of a Vibe Code

Built in 5 Days — Information Systems — Junior — No Prior Python Experience

By Jaden Gomez

> *This project was built in active collaboration with AI as a development tool. All architectural decisions, security validations, threat modeling, and iterative fixes were directed, challenged, and verified manually.*

The Short Version

Built a full-stack banking web application with security-focused design from scratch in ~5 days (3–5 hours/day, around a full class schedule and rugby practice). Prior experience: Java and basic SQL. No Python, no Flask, no web dev, no security background.

The app was scanned with OWASP ZAP DAST with **no high-severity issues identified**, has a live CI/CD pipeline, runs on a real Azure VM behind Nginx with HTTPS, and was audited manually for logic flaws that automated scanners can't detect.

Live domain: bankofvibecode.com

Tech Stack

Layer	Technology
Backend	Python / Flask
WSGI Server	Gunicorn (4 workers)
Database	MySQL
Reverse Proxy	Nginx (HTTPS, HSTS, CSP headers)
SSL	Let's Encrypt
Containerization	Docker
Cloud	Azure VM
CI/CD	GitHub Actions
Security Testing	OWASP ZAP (automated, runs on every push)

Why a Banking App

Intentional. A banking application forces you to think about the problems that actually matter in real systems:

- **Money handling** — floating point arithmetic breaks financial math. `float` is wrong. `Decimal` is right.
- **Concurrency** — two simultaneous withdrawal requests can both pass a balance check before either one writes. Race conditions aren't theoretical.
- **Abuse prevention** — rate limiting, account lockouts, and CSRF protection aren't optional features in a financial context.
- **Security from day one** — building the app *around* the OWASP Top 10 threat model rather than patching it afterward.

Most first projects are todo apps. This one was designed to fail in interesting ways so those failure modes could be understood and addressed.

System Architecture

Request flow through the stack:

User → Nginx → Gunicorn → Flask → MySQL

Nginx handles all incoming traffic, terminates SSL, enforces HSTS, and proxies requests to Gunicorn on port 5000. Gunicorn runs 4 worker processes, meaning concurrent requests are handled without blocking. Flask processes application logic and reads/writes to MySQL. The Flask app itself is never directly exposed to the internet.

On scalability: the current single-VM setup handles moderate traffic without issue. Scaling beyond that would require multiple app instances behind a load balancer and moving session storage to a shared backend like Redis — neither of which is implemented in this version.

Security Design

Authentication & Session Management

- Passwords hashed with Werkzeug (salted bcrypt) — plaintext passwords are never stored

- Per-account lockout after 5 failed attempts (15-minute lockout), independent of IP
- IP-based rate limiting on login (5/min), register (5/min), deposit (10/min), withdraw (10/min)
- Session fixation protection — `session.clear()` before setting session data on login
- Session hardening: `HTTPOnly`, `Secure`, `SameSite=Lax` cookie flags
- 30-minute session timeout

Financial Logic

- **Atomic withdrawal transactions** — single SQL `UPDATE ... WHERE balance >= amount` with `rowcount` check eliminates the Time-of-Check/Time-of-Use race condition
- **`Decimal` throughout** — every money input converted using `Decimal(request.form["amount"])` with `InvalidOperation` handling. No float arithmetic.
- Transaction caps: $10,000 max per deposit/withdrawal
- Overdraft protection enforced at the database level, not the application level

Input & Request Validation

- Parameterized queries throughout — user input treated as data, never as executable SQL
- CSRF protection via Flask-WTF on every form
- Input length caps: full name (100), username (50), password (128)
- Account type validation — only `"checking"` or `"savings"` accepted on financial routes

HTTP Security Headers

- **X-Frame-Options:** DENY
- **X-Content-Type-Options:** nosniff
- **Referrer-Policy:** strict-origin-when-cross-origin
- **Content-Security-Policy:** default-src 'self'; script-src 'self'; object-src 'none'; frame-ancestors 'none'

Infrastructure

- Nginx as reverse proxy — Gunicorn never directly exposed
- HTTPS via Let's Encrypt, HSTS at the Nginx layer
- Docker containerization — app isolated from host environment
- Secrets via environment variables, never in source code

- `.gitignore` prevents `.env` from reaching version control

Logging

- Rotating file logs — startup, logins, failed attempts, deposits, withdrawals
- Real IP logging via `ProxyFix` behind the reverse proxy
- Account lockout events logged separately from standard failed attempts

---

CI/CD Pipeline

Every `git push` to `main`:

1. GitHub Actions connects to the Azure VM via SSH
2. Builds a fresh Docker image
3. Stops and removes the old container
4. Starts the new container with environment variables injected
5. Restarts Nginx
6. Runs an OWASP ZAP baseline scan against the live domain automatically

Security scanning is integrated into the deployment pipeline, not a manual afterthought.

OWASP ZAP Scan Results

Ran OWASP ZAP 2.17.0 against the live application (passive + active baseline scan).

Result: no high-severity issues identified. No confirmed exploits.

ZAP raised one medium-confidence SQL injection flag which was a false positive — the login endpoint returns different responses based on account lockout state, which ZAP interpreted as injection behavior. The underlying query is parameterized, ruling out actual injection.

ZAP confirmed rate limiting was functional (429 responses observed), and identified no XSS vectors, authentication bypasses, or CSRF gaps.

Note: DAST tools like ZAP are effective at catching common vulnerability patterns but cannot detect application-level logic flaws. The findings above reflect automated scan results only.

Manual Audit — What Got Found and Fixed

After building the initial version, the app was audited manually for logic-level vulnerabilities automated tools miss. Two rounds of fixes:

Issue	Status
`/test500` debug route live in production	✅ Removed
Withdrawal race condition (TOCTOU)	✅ Fixed with atomic SQL
`float` used for all money math	✅ Replaced with `Decimal` throughout
Rate limiting only on `/login`	✅ Extended to `/register`, `/deposit`, `/withdraw`
`SELECT *` passing password hash to templates	✅ Fixed — only required columns fetched
No input length limits	✅ Added server-side caps
No deposit amount cap	✅ $10,000 max enforced
IP-only rate limiting bypassable by distributed attack	✅ Added per-account lockout
Session fixation on login	✅ Fixed with `session.clear()`
ZAP pipeline `continue-on-error: true`	✅ Removed — scan now enforces on failure

Known Limitations

This is a learning project and portfolio piece, not a production financial system. Known gaps:

- **Server-side sessions** — current client-side signed cookies can't be invalidated server-side on logout; Redis-backed sessions would fix this
- **CSP** `unsafe-inline` — still present in `style-src`; full nonce implementation is the correct fix but requires auditing all templates first
- **Docker bridge networking** — running `--network host`; a proper bridge network with explicit port exposure is better practice
- **Non-root deployment user** — CI/CD deploys as the default VM user; a limited dedicated user is more appropriate

- **Log sanitization** — usernames logged on failed attempts; hashed identifiers would reduce exposure if logs are exfiltrated

---

What I Learned

Security isn't a feature you add at the end — it's a set of constraints you design around from the start. Tools like OWASP ZAP are useful for catching surface-level issues, but they can't replace understanding *why* a race condition exists or *what* session fixation actually allows an attacker to do.

The most useful shift in thinking was moving from "does this work?" to "how would someone break this?" That question shaped every architectural decision in this project.

Skills Demonstrated

`Python` `Flask` `MySQL` `Docker` `Nginx` `GitHub Actions` `Azure` `OWASP ZAP` `CI/CD` `Threat Modeling` `Parameterized Queries` `CSRF Protection` `Rate Limiting` `Session Security` `Linux` `SSL/TLS` `Gunicorn` `Defensive Programming`